

Car Wash Rush

Game Rules, Mechanics, Economy, Inventory, and Build Roadmap
Version 1.0 - August 7, 2026

Design Thesis

Car Wash Rush is a city sandbox tycoon with chaotic simulator moments. The surface experience should be simple, funny, and immediately readable. Underneath, the game should gradually expose real business forces: location, price, quality, demand, competition, employee reliability, offline events, and expansion.

Current prototype	Greybox starter corner, dirty car, dirt prompts, temporary coins, sponge upgrade.
Next technical priority	Replace temporary player values with a real inventory/profile model, still in memory first, then DataStore persistence.
Launch product stance	No pay-to-win. Monetize cosmetics, convenience, server-wide boosts, private servers, subscriptions, and light non-competitive boosts.
Scope warning	Do not build the giant city first. Build durable systems that can scale into the city.

1. Product Definition

Car Wash Rush starts the player with almost nothing: a rag, a bucket, and a dirty vehicle at a starter intersection. The player earns cash, upgrades tools, controls locations, hires workers, automates wash stations, competes with rivals, and expands across city and rural districts.

The game should become the first entry in a repeatable studio format: small entrepreneurial activity to large empire. Future games can reuse the same progression, economy, inventory, quest, monetization, and analytics patterns.

Core design pillars

- **Instant clarity:** a first-time player must understand the first wash action in less than 30 seconds.
- **Hands-on first:** manual washing teaches value before automation takes over.
- **Chaos with rules:** foam parties, mud rushes, and rival behavior are funny, but the player always knows the next useful action.
- **Economics underneath:** price, quality, demand, supply, location, and competition shape profit.
- **Mobile first:** prompts, UI, camera, and timing must work on phone and tablet.
- **No pay-to-win:** paid features can save time or look better, but should not create unbeatable paid-only progression.

2. Player Lifecycle

The game should guide players through increasingly complex business systems. The first play session is controlled; later sessions open up city exploration and competition.

Stage	Player Experience	Primary Systems
First 2 minutes	Spawn at starter corner, clean first car, earn first coins.	Proximity prompts, dirt patches, coins, simple car spawn.
First 20 minutes	Unlock sponge, hose, first helper, price choice, and first meaningful location decision.	Inventory, upgrades, tutorial state, starter demand model.
First 1-2 hours	Reach first owned car wash location and basic automation.	Location ownership, employees, offline earnings, station upgrades.
First week	Compete across locations, respond to offline events, unlock new district.	District map, rivals, quests, event scheduler.
Long term	Expand to city, rural, industrial, beach, and racing districts.	Prestige, cosmetics, live ops, subscriptions, analytics.

3. Core Loop Rules

The first loop must stay clean and deterministic.

- 1 A dirty vehicle arrives at a wash spot.
- 2 The player uses the currently equipped tool on visible dirt patches.
- 3 Each patch requires a prompt hold or future tool action.

- 4 When all required dirt patches are cleaned, the car pays the player.
- 5 The player spends coins on tools, stations, employees, or location improvements.
- 6 Upgrades increase speed, capacity, quality, customer types, idle performance, or cosmetics.
- 7 The city responds through demand changes, rivals, events, and location opportunities.

Prototype rule

Until inventory and saving are correct, do not add more economy depth. Bad data models are expensive to unwind later.

4. Cleaning Mechanics

Cleaning should evolve from simple prompt holds into tool-specific actions. Each vehicle has a generated dirt profile, and each dirt type prefers certain tools.

Dirt Type	Early Tool	Advanced Tool	Gameplay Purpose
Dust	Rag	Microfiber mitt	Fast starter feedback.
Mud	Sponge	Pressure washer	Creates satisfying visual removal.
Grease	Soap bucket	Degreaser	Requires specialized purchase.
Paint splatter	Brush	Polish buffer	Slower premium task.
Farm grime	Hose	Heavy washer	Rural district identity.
Race grime	Foam sprayer	Detailing station	Racing district identity.
Mystery goo	Event tool	Event station	Limited-time chaos.

Cleaning feedback rules

- Dirt should visibly shrink, fade, pop, or wash off.
- Every complete car should have a clear reward moment.
- A better tool must feel better immediately: shorter hold time, bigger effect, better sound, larger splash, or higher reward quality.
- Avoid making the player hunt tiny dirt pixels. The game should be readable on mobile.

5. Inventory and Player Profile Rules

Inventory is the next system to code. The current prototype only stores temporary session values on the Player instance. That is fine for a smoke test, but it is not enough for a real tycoon.

Roblox DataStoreService is the platform service for data that must persist between sessions, including inventory and skill-like progression. The game should use server-owned profile data and later save it through DataStoreService.

Data Group	Fields	Rules
Currencies	coins, gems, eventTickets	Server authoritative. Client can request actions but never directly set balances.
Tools	ownedTools, equippedTool, toolLevels	A player may equip one cleaning tool at a time in Phase 1.
Stations	ownedStations, stationLevels, activeLocationId	Stations belong to a location, not globally to the player.
Locations	ownedLocations, claimedIntersections, permits	Claims can expire; permanent owned lots persist.
Employees	employees, employeeLevels, morale, assignedStation	Employees drive idle income and offline events.
Cosmetics	ownedCosmetics, equippedCosmetics	Cosmetics never gate core earning power.
Progress	tutorialStep, quests, unlockedDistricts, prestige	Progression gates UI complexity and map access.
Receipts	processedPurchasesIds	Developer product grants must be idempotent.

Inventory invariants

- All grants happen on the server.
- Every item has a stable string id, for example **tool_rag**, **tool_sponge**, **station_basic_pad**.
- The profile owns ids; config tables define item stats.
- Purchasing an already-owned one-time item should not double-grant it.
- Equipping requires ownership.
- Saving should be versioned from the beginning, for example **schemaVersion = 1**.

6. Tool Progression

Tool	Unlock	Main Effect	Design Note
Old rag	Default	Cleans dust slowly.	Teaches the action.
Sponge	50 coins	Faster basic cleaning and small bonus.	First upgrade; must arrive quickly.
Hose	150 coins	Handles mud better.	Adds range and water feedback.
Soap sprayer	400 coins	Pre-treats dirt groups.	Bridge to stations.
Pressure washer	1,000 coins	Fast mud and farm grime.	Strong midgame satisfaction.
Foam cannon	Event or 2,500 coins	Large area cleaning and cosmetic foam.	Primary viral tool.
Detail buffer	5,000 coins	High-quality finish and VIP tips.	Introduces quality stat.

7. Vehicles and Customers

Vehicles should be mechanically distinct. A compact car is fast and cheap; a tractor is slow and high-ticket; a race bike is rare and socially exciting.

Vehicle Class	Where It Appears	Dirt Bias	Economic Role
Compact	Starter streets	Dust	Fast onboarding volume.
Delivery van	City and strip centers	Dust, grease	Reliable mid-ticket job.
SUV	Suburbs	Mud, dust	Normal family customer.
Farm tractor	Rural	Farm grime, mud	Low traffic, high payout.
Construction truck	Industrial	Mud, grease	Tool specialization.
Luxury car	Downtown	Dust, polish	High elasticity tolerance.
Fictional race bike	Racing district/events	Race grime	Rare VIP event and cosmetic hook.

8. Locations, Territory, and Map Expansion

The map should feel large over time, but launch development should be district-based. Each district unlocks new economic conditions, vehicles, and player decisions.

District	Traffic	Competition	Specialty
Starter block	Low	Low	Tutorial and first price decisions.
Downtown	High	High	Price wars, luxury customers, high rent.
Suburbs	Medium	Medium	Stable demand and family vehicles.
Strip centers	Medium	Medium-high	Shared parking lots and rival crowding.
Industrial	Low-medium	Medium	Grease, trucks, contracts.
Rural	Low	Low-medium	Tractors, farm equipment, high mud jobs.
Racing district	Event-driven	High	Fictional race vehicles and premium cosmetics.

Territory rules

- Street corners can be claimed temporarily, not permanently.
- Owned lots are permanent progression assets.
- Other players can stand near a controlled intersection, but the controller receives demand priority.
- Undercutting can steal some demand but reduces profit and can trigger price wars.
- Better quality and reputation allow higher prices without losing as many customers.

9. Economics Model

The game should use simplified economics that players can feel, even if they do not name the concepts.

Concept	Game Translation	Player-Facing Result
Demand	Traffic and customer count by location and time.	Busy corners matter.
Supply	Number of active washers serving the same area.	Too many players lower profit.
Elasticity	Customer willingness to accept higher prices.	Luxury customers tolerate more; commuters leave faster.
Competition	Rivals set prices and quality nearby.	Price wars and relocation decisions.
Monopoly	Owning several nearby locations improves control.	Pricing power, but more events and scrutiny.
Quality	Tool/station/service score.	Higher prices and VIP tips.
Reputation	Long-term customer trust by district.	More stable demand and better customers.

Phase 1 economy rule

Only implement one economic equation at first: demand falls when price is too high for a location. Do not implement full competition, monopoly, and elasticity until profile/inventory is stable.

10. Offline Events

The offline system should be sticky without feeling purely punitive. The player should return to urgent business decisions, not just damage.

Event	Chance Basis	Effect	Good Design Version
Power outage	Hourly offline roll	Station stops temporarily.	Pay repair, wait, or use backup upgrade.
Employee quit	Morale and wage risk	Worker unavailable.	Rehire, raise wages, or train replacement.
Rival pressure	Competitive intersections	Demand share drops.	Lower price, improve quality, or move.
Territory challenge	Unowned street corners	Claim weakens.	Return to defend or accept lower income.
Equipment jam	Station age and usage	Reduced speed.	Repair or upgrade.
Surprise rush	Positive offline roll	Bonus queue waiting.	Come back to cash in.
Hybrid rule Every offline hour can roll a business event, but hard losses should be capped. The goal is high return urgency, not rage-quitting.			

11. Employees and Automation

- Employees convert owned stations into idle income.
- Employees have speed, reliability, morale, and assigned station.

- Low morale increases quit chance and mistakes.
- Training improves output but costs coins.
- Automation should never remove all reasons to actively play; active play should remain the best income per minute.

12. Monetization Rules

Roblox passes are appropriate for one-time privileges; developer products are appropriate for repeat purchases. MarketplaceService purchase grants must be validated and handled server-side.

Product Type	Allowed Examples	Restriction
Game pass	VIP lounge, auto collect tips, extra cosmetic loadouts.	Avoid permanent paid-only best earning path.
Developer product	Server foam party, temporary rush event, instant clean tokens.	Receipt granting must be idempotent.
Subscription	Cosmetics, idle cap, 25% idle boost, higher supercar chance.	Do not dominate active competitive leaderboards.
Private server	Creator filming, friend sessions, custom event controls.	No exclusive economy advantage in public progression.
Cosmetics	Foam colors, tool skins, signs, employee uniforms.	Best monetization category for no-pay-to-win.

13. Analytics and Tuning

Roblox custom events can track the specific funnels that matter to this game: onboarding, progression, economy, and shop behavior.

Metric	Target Question
Time to first wash	Do players understand the game immediately?
Time to first upgrade	Is the first reward fast enough?
Cars cleaned per session	Is the core loop sticky?
Upgrade purchase distribution	Where does progression stall?
Offline return rate	Do offline events bring players back?
Price changes by location	Are economics understandable?
Shop views to purchases	Is monetization clear without pressure?
D1/D7 retention	Is the game worth returning to?

14. Current Prototype Assessment

System	Status	Comment
Greybox world	Working	Starter corner is generated from server code.
Cleaning	Working prototype	Dirt patches use ProximityPrompt.
Currency	Temporary	Coins exist only as session leaderstats.
Inventory	Not real yet	Current tool is a single StringValue, not an inventory model.
Saving	Not implemented	No DataStore persistence yet.
Upgrades	Temporary	Sponge is hard-coded, not config-driven inventory.
Economics	Not implemented	No demand, price, rivals, or location ownership yet.
Monetization	Not implemented	Do not add until profile and receipt handling are ready.

15. Next Thing To Code

Recommendation

Code the inventory/profile layer next. Do this before adding more gameplay features. The game needs a stable server-owned data shape for tools, currencies, upgrades, tutorial progress, and later DataStore saving.

Step-by-step implementation order

- 1 Create a shared item catalog in `src/shared/Catalog.luau` with tool ids, names, costs, and stats.
- 2 Replace `Tool = 'Rag'` with `ownedTools`, `equippedToolId`, and `toolLevels` in `PlayerData`.
- 3 Expose server functions: `ownsTool`, `grantTool`, `equipTool`, `buyTool`, and `getEquippedToolConfig`.
- 4 Update cleaning rewards and hold durations to read from equipped tool config.
- 5 Update the sponge upgrade prompt to call `buyTool(player, 'tool_sponge')`.
- 6 Add a simple debug print or temporary UI state so we can confirm inventory changes during Play mode.
- 7 Only after in-memory inventory works, add DataStore load/save with a schema version.

Acceptance tests for the next code change

- New player starts owning only `tool_rag`.
- New player has `tool_rag` equipped.
- Player cannot equip `tool_sponge` before buying it.
- Player cannot buy sponge without enough coins.
- Buying sponge deducts coins once and grants ownership once.
- Buying sponge again does not double-charge or duplicate it.
- Equipped sponge changes cleaning speed or reward.

16. Platform Notes and Sources

Official Roblox docs checked August 7, 2026:

- Data stores: <https://create.roblox.com/docs/cloud-services/data-stores>
- DataStoreService reference: <https://create.roblox.com/docs/reference/engine/classes/DataStoreService>
- Proximity prompts: <https://create.roblox.com/docs/ui/proximity-prompts>
- Passes: <https://create.roblox.com/docs/production/monetization/passes>
- Developer products: <https://create.roblox.com/docs/production/monetization/developer-products>
- Custom analytics events: <https://create.roblox.com/docs/production/analytics/custom-events>

Implementation note: this document defines design rules and next engineering steps. It is not a legal, tax, or platform-policy guarantee. Re-check Roblox docs before adding monetization, paid random rewards, ads, or external brand deals.